

Paradigmi di Programmazione - A.A. 2022-23

Esame Scritto del 15/06/2023

CRITERI DI VALUTAZIONE:

La prova è superata se si ottengono almeno 12 punti negli esercizi 1,2,3 e almeno 18 punti complessivamente.

Esercizio 1 [Punti 4]

Applicare la β -riduzione alla seguente λ -espressione applicando la strategia **call-by-value** fino a raggiungere una espressione non ulteriormente riducibile o ad accorgersi che la derivazione è infinita. Ripetere l'esercizio applicando la strategia **call-by-name**.

$$(\lambda f. \lambda x. x)((\lambda f. f f)(\lambda x. x x))$$

Nella soluzione, mostrare tutti i passi di riduzione calcolati, sottolineando ad ogni passo la porzione di espressione a cui si applica la β -riduzione (redex) ed evidenziando le eventuali α -conversioni.

Esercizio 2 [Punti 4]

Indicare il tipo della seguente funzione OCaml, mostrando i passi fatti per inferirlo:

```
fun x -> fun y -> match x with
  | [] -> []
  | z::[] -> z::z::[]
  | (z1,z2)::x' -> (z1+y,z2)::x';;
```

Esercizio 3 [Punti 7]

Definire in OCaml le funzioni `stessalunghezza` e `separa` con i seguenti tipi

```
stessalunghezza : 'a list list -> bool
separa : ('a * 'b) list list -> 'a list list * 'b list list
```

tali che:

- `stessalunghezza` prende una lista di liste `lis` e verifica se tutte le liste contenute in `lis` hanno la stessa lunghezza. Ad esempio:

```
stessalunghezza [[1;2;3];[4;5;6];[7;8;9]]      (* restituisce true *)
stessalunghezza [['a';'b'];['c']]              (* restituisce false *)
```

- `separa` prende una lista di liste di coppie `lis` e restituisce una coppia di liste di liste (`lis1, lis2`) tale che ogni lista di coppie $[(a_1, b_1); \dots; (a_N, b_N)]$ in `lis` viene trasformata in due liste, una con i primi elementi $[a_1; \dots; a_N]$ e una con i secondi elementi $[b_1; \dots; b_N]$, e le due liste vengono inserite rispettivamente in `lis1` e `lis2`. Ad esempio:

```
separa [[(1,'a'); (2,'b'); (3,'c')]; [(4,'d'); (5,'e')]]
(* restituisce ([1;2;3];[4;5]), [['a','b','c'], ['d','e']] *)
```

Esercizio 4 [Punti 15]

Si estenda il linguaggio MiniCaml visto a lezione con il costrutto `NaryFun` per la definizione di *funzioni anonime n-arie* (non Curried), ossia che prevedono un numero arbitrario di parametri formali. Queste nuove funzioni saranno applicate tramite un nuovo costrutto di applicazione che, data una lista di parametri attuali con una lunghezza pari al numero di parametri formali, lega ogni parametro formale al corrispondente parametro attuale per creare l'ambiente in cui viene eseguito il corpo della funzione. Ad esempio (usando una sintassi simile a OCaml):

```
nary_fun [x,y] -> x+y      (* definisce una funzione che prende x e y, e ne fa la somma *)
```

```
(nary_fun [x,y] -> x+y) [3,2]  (* applica la funzione di somma a 3 e 2 *)
```

Si mostri come deve essere modificato l'interprete OCaml del linguaggio.